

LAPORAN PRAKTIKUM JOBSHEET 13
TREE



Disusun Oleh:

ALYA FAKHRUN NISA

254107060036

SIB 1A

**PROGRAM STUDI D4 SISTEM
INFORMASI BISNIS JURUSAN
TEKNOLOGI INFORMASI
POLITEKNIK
NEGERI
MALANG 2026**

Link Github : https://github.com/alyafakhrunnisa/Jobsheet14_Tree

14.2.1 Percobaan 1

a. Class Mahasiswa

```
1 package Pertemuan14;
2
3 public class Mahasiswa02 {
4     String nim;
5     String nama;
6     String kelas;
7     double ipk;
8
9     public Mahasiswa02(){
10    }
11
12    public Mahasiswa02(String nim, String nama, String kelas, double ipk) {
13        this.nim = nim;
14        this.nama = nama;
15        this.kelas = kelas;
16        this.ipk = ipk;
17    }
18
19    public void tampilInformasi(){
20        System.out.println("NIM: " +this.nim+ " " +
21        "Nama: " +this.nama+ " " +
22        "Kelas: " +this.kelas+ " " +
23        "IPK: " +this.ipk);
24    }
25 }
```

b. Class Node

```
1 package Pertemuan14;
2
3 public class Node02 {
4     Mahasiswa02 mahasiswa;
5     Node02 left, right;
6
7     public Node02(){
8     }
9
10    public Node02(Mahasiswa02 Mahasiswa) {
11        this.mahasiswa = Mahasiswa;
12        left = right = null;
13    }
14 }
```

c. Class BinaryTree

```
package Pertemuan14;

public class BinaryTree02 {
    Node02 root;

    public BinaryTree02() {
        root = null;
    }

    public boolean isEmpty() {
        return root == null;
    }

    public void add(Mahasiswa02 mahasiswa) {
        Node02 newNode = new Node02(mahasiswa);
        if (isEmpty()) {
            root = newNode;
        } else {
            Node02 current = root;
            Node02 parent = null;
            while (true) {
                parent = current;
                if (mahasiswa.ipk < current.mahasiswa.ipk) {
                    current = current.left;
                    if (current == null) {
                        parent.left = newNode;
                        return;
                    }
                } else {
                    current = current.right;
                    if (current == null) {
                        parent.right = newNode;
                        return;
                    }
                }
            }
        }
    }

    boolean find(double ipk) {
        boolean result = false;
        Node02 current = root;
        while (current != null) {
            if (current.mahasiswa.ipk == ipk) {
                result = true;
                break;
            } else if (ipk > current.mahasiswa.ipk) {
                current = current.right;
            } else {
                current = current.left;
            }
        }
        return result;
    }

    void traversePreOrder(Node02 node) {
        if (node != null) {
            node.mahasiswa.tampilInformasi();
            traversePreOrder(node.left);
            traversePreOrder(node.right);
        }
    }

    void traverseInOrder(Node02 node) {
        if (node != null) {
            traverseInOrder(node.left);
            node.mahasiswa.tampilInformasi();
            traverseInOrder(node.right);
        }
    }

    void traversePostOrder(Node02 node) {
        if (node != null) {
            traversePostOrder(node.left);
            traversePostOrder(node.right);
            node.mahasiswa.tampilInformasi();
        }
    }
}
```

```

Node02 getSuccessor(Node02 del) {
    Node02 successor = del.right;
    Node02 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

void delete(double ipk) {
    if (isEmpty()) {
        System.out.println(x: "Binary tree kosong");
        return;
    }
    // cari node (current) yang akan dihapus
    Node02 parent = root;
    Node02 current = root;
    boolean isLeftChild = false;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            break;
        } else if (ipk < current.mahasiswa.ipk) {
            parent = current;
            current = current.left;
            isLeftChild = true;
        } else if (ipk > current.mahasiswa.ipk) {
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }
    if (current == null) {
        System.out.println(x: "Data tidak ditemukan");
        return;
    } else {
        // jika tidak ada anak (leaf), maka node dihapus
        if (current.left == null && current.right == null) {
            if (current == root) {
                root = null;
            } else {
                if (isLeftChild) {
                    parent.left = null;
                } else {
                    parent.right = null;
                }
            }
        } else if (current.left == null) { // jika hanya punya 1 anak (kanan)
            if (current == root) {
                root = current.right;
            } else {
                if (isLeftChild) {
                    parent.left = current.right;
                } else {
                    parent.right = current.right;
                }
            }
        } else if (current.right == null) { // jika hanya punya 1 anak (kiri)
            if (current == root) {
                root = current.left;
            } else {
                if (isLeftChild) {
                    parent.left = current.left;
                } else {
                    parent.right = current.left;
                }
            }
        } else { // jika punya 2 anak
            Node02 successor = getSuccessor(current);
            System.out.println(x: "Jika 2 anak, current = ");
            successor.mahasiswa.tampilInformasi();
            if (current == root) {
                root = successor;
            } else {
                if (isLeftChild) {
                    parent.left = successor;
                } else {
                    parent.right = successor;
                }
            }
            successor.left = current.left;
        }
    }
}

```

d. Class BinaryTreeMain

```

import java.util.Scanner; // The import java.util.Scanner is never used

public class BinaryTreeMain02 {
    Run/Debug
    public static void main(String[] args) {
        BinaryTree02 bst = new BinaryTree02();
        bst.add(new Mahasiswa02(nim: "244160121", nama: "Ali", kelas: "A", ipk: 3.57));
        bst.add(new Mahasiswa02(nim: "244160221", nama: "Badar", kelas: "B", ipk: 3.85));
        bst.add(new Mahasiswa02(nim: "244160185", nama: "Candra", kelas: "C", ipk: 3.21));
        bst.add(new Mahasiswa02(nim: "244160220", nama: "Dewi", kelas: "B", ipk: 3.54));

        System.out.println(x: "\nDaftar semua mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);

        System.out.println(x: "\nPencarian data mahasiswa:");
        System.out.print(s: "Cari mahasiswa dengan ipk: 3.54 : ");
        String hasilCari = bst.find(ipk: 3.54) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);

        System.out.print(s: "Cari mahasiswa dengan ipk: 3.22 : ");
        hasilCari = bst.find(ipk: 3.22) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);

        bst.add(new Mahasiswa02(nim: "244160131", nama: "Dewi", kelas: "A", ipk: 3.72));
        bst.add(new Mahasiswa02(nim: "244160205", nama: "Ehsan", kelas: "D", ipk: 3.37));
        bst.add(new Mahasiswa02(nim: "244160170", nama: "Fizi", kelas: "B", ipk: 3.46));
        System.out.println(x: "\nDaftar semua mahasiswa setelah penambahan 3 mahasiswa:");
        System.out.println(x: "InOrder Traversal:");
        bst.traverseInOrder(bst.root);
        System.out.println(x: "\nPreOrder Traversal:");
        bst.traversePreOrder(bst.root);
        System.out.println(x: "\nPostOrder Traversal:");
        bst.traversePostOrder(bst.root);

        System.out.println(x: "\nPenghapusan data mahasiswa:");
        bst.delete(ipk: 3.57);
        System.out.println(x: "\nDaftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);
    }
}

```

e. Output

```

Daftar semua mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

Pencarian data mahasiswa:
Cari mahasiswa dengan ipk: 3.54 : Ditemukan
Cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:
InOrder Traversal:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Dewi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

PreOrder Traversal:
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160131 Nama: Dewi Kelas: A IPK: 3.72

PostOrder Traversal:
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160131 Nama: Dewi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57

Penghapusan data mahasiswa
Jika 2 anak, current =
NIM: 244160131 Nama: Dewi Kelas: A IPK: 3.72

```

```

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160131 Nama: Dewi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

```

14.2.2 Pertanyaan Percobaan

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?

Jawab :

Dalam Binary Search Tree (BST), setiap node memiliki aturan terurut: semua node di subtree kiri memiliki nilai lebih kecil, dan semua node di subtree kanan memiliki nilai lebih besar dari node tersebut. Karena aturan ini, setiap langkah pencarian langsung membuang setengah dari sisa tree (seperti binary search). Kompleksitas pencarian BST adalah $O(\log n)$ untuk tree yang seimbang, sedangkan Binary Tree biasa tidak memiliki aturan urutan sehingga harus mencari ke semua node $\rightarrow O(n)$.

2. Untuk apakah di class **Node**, kegunaan dari atribut **left** dan **right**?

Jawab :

Atribut **left** dan **right** pada class **Node02** merupakan referensi (pointer) ke node anak:

- **left** : menunjuk ke node anak kiri, yaitu node yang memiliki IPK lebih kecil dari node saat ini.
- **right** : menunjuk ke node anak kanan, yaitu node yang memiliki IPK lebih besar dari node saat ini.

3. a. Untuk apakah kegunaan dari atribut **root** di dalam class **BinaryTree**?

Jawab :

Atribut **root** menyimpan referensi ke node paling atas (akar) dari seluruh tree. Tanpa **root**, kita tidak bisa mengakses tree sama sekali karena tidak ada titik masuk. Semua operasi (**add**, **find**, **delete**, **traverse**) dimulai dari **root**.

b. Ketika objek tree pertama kali dibuat, apakah nilai dari **root**?

Jawab :

Saat konstruktor **BinaryTree02()** dipanggil, nilai **root** diinisialisasi dengan **null**, yang berarti tree masih kosong dan belum ada node satu pun.

4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?

Jawab :

Karena **root == null** (tree kosong), node baru langsung dijadikan sebagai **root** dari tree. Tidak ada perbandingan IPK yang dilakukan karena tidak ada node lain sebagai pembanding.

5. Perhatikan method **add()**, di dalamnya terdapat baris program seperti di bawah ini.

Jelaskan secara detail untuk apa baris program tersebut?

```
parent = current;
if (mahasiswa.ipk < current.mahasiswa.ipk) {
    current = current.left;
    if (current == null) {
        parent.left = newNode;
        return;
    }
} else {
    current = current.right;
    if (current == null) {
        parent.right = newNode;
        return;
    }
}
```

Jawab :

- **parent = current;** Menyimpan node saat ini sebagai **parent** sebelum kita bergeser turun ke level di bawahnya. Ini penting agar kita tidak kehilangan jejak node terakhir yang valid.

- **if (mahasiswa.ipk < current.mahasiswa.ipk):** Memeriksa apakah IPK mahasiswa baru lebih kecil dari IPK node saat ini.

- **current = current.left;** Jika lebih kecil, kita bergeser ke anak kiri.
- **if (current == null):** Jika anak kiri ternyata kosong (bernilai **null**), berarti posisi yang tepat telah ditemukan.
- **parent.left = newNode; return;** Node baru dimasukkan dan dihubungkan menjadi anak kiri dari **parent**, lalu fungsi selesai.

- **else:** Jika IPK mahasiswa baru lebih besar atau sama dengan IPK node saat ini.

- **current = current.right;** Kita bergeser ke anak kanan.
- **if (current == null):** Jika anak kanan kosong, posisi telah ditemukan.
- **parent.right = newNode; return;** Node baru dimasukkan dan dihubungkan menjadi anak kanan dari **parent**, lalu fungsi selesai.

6. Jelaskan langkah-langkah pada method **delete()** saat menghapus sebuah node yang memiliki dua anak. Bagaimana method **getSuccessor()** membantu dalam proses ini?

Jawab :

Langkah-langkah delete() untuk node dengan 2 anak:

1. Cari node yang akan dihapus (current) sambil mencatat parent-nya dan apakah ia anak
2. Deteksi bahwa current punya 2 anak (current.left != null && current.right != null).
3. Panggil getSuccessor(current) untuk menemukan pengganti (successor).
4. Gantikan posisi current dengan successor (hubungkan parent ke successor).
5. Sambungkan subtree kiri current ke successor.left.

Peran getSuccessor():

Method ini mencari in-order successor yaitu node dengan nilai terkecil di subtree kanan dari node yang akan dihapus (node paling kiri di subtree kanan). Successor ini dipilih karena:

- Nilainya lebih besar dari semua node di subtree kiri current → aturan BST tetap terpenuhi.
- Nilainya lebih kecil dari semua node lain di subtree kanan current → aturan BST tetap terpenuhi.

Dalam getSuccessor():

- Mulai dari del.right, terus ke kiri (successor = successor.left) sampai tidak bisa lagi.
- Jika successor bukan langsung anak kanan: successorParent.left = successor.right (putuskan successor dari posisi lama), lalu successor.right = del.right (sambungkan subtree kanan lama).
- Kembalikan successor agar bisa menggantikan node yang dihapus.

14.3.1 Tahapan Percobaan

a. Class BinaryTreeArray

```
package Pertemuan14;
public class BinaryTreeArray02 {
    Mahasiswa02[] dataMahasiswa;
    int idxLast;

    public BinaryTreeArray02() {
        this.dataMahasiswa = new Mahasiswa02[10];
    }

    void populateData(Mahasiswa02 dataMhs[], int idxLast) {
        this.dataMahasiswa = dataMhs;
        this.idxLast = idxLast;
    }

    void traverseInOrder(int idxStart) {
        if (idxStart <= idxLast) {
            if (dataMahasiswa[idxStart] != null) {
                traverseInOrder(2 * idxStart + 1);
                dataMahasiswa[idxStart].tampilInformasi();
                traverseInOrder(2 * idxStart + 2);
            }
        }
    }
}
```

b. Class BinaryTreeArrayMain

```
package Pertemuan14;
import java.util.Scanner;
public class BinaryTreeArrayMain02 {
    public static void main(String[] args){
        BinaryTreeArray02 bta = new BinaryTreeArray02();
        Mahasiswa02 mhs1 = new Mahasiswa02(nim: "244160121", nama: "Ali", kelas: "A", ipk: 3.57);
        Mahasiswa02 mhs2 = new Mahasiswa02(nim: "244160185", nama: "Candra", kelas: "C", ipk: 3.41);
        Mahasiswa02 mhs3 = new Mahasiswa02(nim: "244160221", nama: "Badar", kelas: "B", ipk: 3.75);
        Mahasiswa02 mhs4 = new Mahasiswa02(nim: "244160220", nama: "Dewi", kelas: "B", ipk: 3.35);
        Mahasiswa02 mhs5 = new Mahasiswa02(nim: "244160131", nama: "Devi", kelas: "A", ipk: 3.48);
        Mahasiswa02 mhs6 = new Mahasiswa02(nim: "244160205", nama: "Ehsan", kelas: "D", ipk: 3.61);
        Mahasiswa02 mhs7 = new Mahasiswa02(nim: "244160170", nama: "Fizi", kelas: "B", ipk: 3.86);

        Mahasiswa02[] dataMahasiswas = {mhs1, mhs2, mhs3, mhs4, mhs5, mhs6, mhs7, null, null, null};
        int idxLast = 6;
        bta.populateData(dataMahasiswas, idxLast);
        System.out.println(x: "\nInorder Traversal Mahasiswa: ");
        bta.traverseInOrder(idxStart: 0);
    }
}
```

c. Output

```
Inorder Traversal Mahasiswa:
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86
```

14.3.2 Pertanyaan Percobaan

1. Apakah kegunaan dari atribut data dan idxLast yang ada di class **BinaryTreeArray**?

Jawab :

- dataMahasiswa[]: Array yang menyimpan seluruh node (objek Mahasiswa) dalam binary tree. Posisi indeks array merepresentasikan posisi node dalam tree (indeks 0 = root, indeks 1 = anak kiri root, indeks 2 = anak kanan root, dst).
- idxLast: Menyimpan indeks terbesar yang terisi data dalam array. Digunakan sebagai batas akhir traversal rekursif agar tidak menelusuri indeks yang kosong secara sia-sia.

2. Apakah kegunaan dari method **populateData()**?

Jawab :

Method populateData() digunakan untuk mengisi/menginisialisasi array dataMahasiswa dengan data dari luar (parameter). Method ini menerima array Mahasiswa dan indeks terakhir, lalu menyalinnya ke atribut class. Ini adalah cara mudah untuk sekaligus memasukkan banyak data ke dalam tree berbasis array tanpa harus memanggil add() satu per satu.

3. Apakah kegunaan dari method **traverseInOrder()**?

Jawab :

Method traverseInOrder(int idxStart) digunakan untuk mengunjungi dan menampilkan semua node dalam binary tree secara In-Order (Kiri → Root → Kanan) menggunakan rekursi.

Jadi, hasilnya adalah tampilan data yang terurut dari IPK terkecil ke terbesar, karena sifat in-order traversal pada BST menghasilkan urutan yang ascending.

4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?

Jawab:

Rumus posisi dalam array:

- Left child dari indeks $i = 2*i + 1$
- Right child dari indeks $i = 2*i + 2$

Untuk indeks 2:

- Left child = $2*2 + 1 =$ indeks 5
- Right child = $2*2 + 2 =$ indeks 6

5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?

Jawab :

Statement `int idxLast = 6` menetapkan bahwa data terakhir yang valid dalam array ada di indeks 6 (dari total 7 data: indeks 0 sampai 6). Nilai ini dipakai dalam method `traverseInOrder()` sebagai kondisi batas rekursi: `if (idxStart <= idxLast)` agar rekursi berhenti dan tidak memproses indeks kosong/null di luar data.

6. Mengapa indeks $2*idxStart+1$ dan $2*idxStart+2$ digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?

Jawab :

Ini merupakan rumus matematika pemetaan binary tree ke array. Maka dengan rumus ini, seluruh struktur hierarki pohon biner dapat direpresentasikan dalam array linear 1D tanpa menggunakan pointer. Setiap level tree menempati posisi berurutan dalam array. Level 0 (root): indeks 0; Level 1: indeks 1-2; Level 2: indeks 3-6; dst. Rumus $2*i+1$ dan $2*i+2$ memungkinkan kita "menavigasi" tree seperti linked list, tetapi hanya menggunakan indeks array biasa.

14.4 Tugas Praktikum

Waktu pengerjaan: 150 menit

1. Buat method di dalam class **BinaryTree00** yang akan menambahkan node dengan cara rekursif (**addRekursif()**).

Jawab :

```
// TUGAS 1
public void addRekursif(Mahasiswa02 mahasiswa) {
    root = addRekursifHelper(root, mahasiswa);
}

private Node02 addRekursifHelper(Node02 current, Mahasiswa02 mahasiswa) {
    if (current == null) {
        return new Node02(mahasiswa);
    }
    if (mahasiswa.ipk < current.mahasiswa.ipk) {
        current.left = addRekursifHelper(current.left, mahasiswa);
    } else {
        current.right = addRekursifHelper(current.right, mahasiswa);
    }
    return current;
}

System.out.println(x: "\n--- Tugas 1: addRekursif ---");
BinaryTree02 bstRekursif = new BinaryTree02();
bstRekursif.addRekursif(new Mahasiswa02(nim: "244160121", nama: "Ali", kelas: "A", ipk: 3.57));
bstRekursif.addRekursif(new Mahasiswa02(nim: "244160221", nama: "Badar", kelas: "B", ipk: 3.85));
bstRekursif.addRekursif(new Mahasiswa02(nim: "244160185", nama: "Candra", kelas: "C", ipk: 3.21));
bstRekursif.addRekursif(new Mahasiswa02(nim: "244160220", nama: "Dewi", kelas: "B", ipk: 3.54));
bstRekursif.addRekursif(new Mahasiswa02(nim: "244160131", nama: "Dewi", kelas: "A", ipk: 3.72));
bstRekursif.addRekursif(new Mahasiswa02(nim: "244160205", nama: "Ehsan", kelas: "D", ipk: 3.37));
bstRekursif.addRekursif(new Mahasiswa02(nim: "244160170", nama: "Fizi", kelas: "B", ipk: 3.46));
System.out.println(x: "InOrder setelah addRekursif:");
bstRekursif.traverseInOrder(bstRekursif.root);

--- Tugas 1: addRekursif ---
InOrder setelah addRekursif:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Dewi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
```

2. Buat method di dalam class **BinaryTree00** untuk menampilkan data mahasiswa dengan IPK paling kecil dan IPK yang paling besar (**cariMinIPK()** dan **cariMaxIPK()**) yang ada di dalam binary search tree.

Jawab :

```
public void cariMaxIPK() {
    if (isEmpty()) {
        System.out.println(x: "Binary tree kosong");
        return;
    }
    // Di BST, nilai terbesar ada di node paling kanan
    Node02 current = root;
    while (current.right != null) {
        current = current.right;
    }
    System.out.println(x: "Mahasiswa dengan IPK terbesar:");
    current.mahasiswa.tampilInformasi();
}

// Tugas 2: cariMinIPK dan cariMaxIPK
System.out.println(x: "\n--- Tugas 2: cariMinIPK dan cariMaxIPK ---");
bstRekursif.cariMinIPK();
bstRekursif.cariMaxIPK();

--- Tugas 2: cariMinIPK dan cariMaxIPK ---
Mahasiswa dengan IPK terkecil:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
Mahasiswa dengan IPK terbesar:
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
```



```

public void tampilMahasiswaIPKdiAtas(double ipkBatas) {
    System.out.println("Mahasiswa dengan IPK di atas " + ipkBatas + ":");
    tampilIPKdiAtasHelper(root, ipkBatas);
}

private void tampilIPKdiAtasHelper(Node02 node, double ipkBatas) {
    if (node == null) return;
    // Jika IPK node > ipkBatas, cari juga di subtree kiri (mungkin ada yg lebih
    tampilIPKdiAtasHelper(node.left, ipkBatas);
    if (node.mahasiswa.ipk > ipkBatas) {
        node.mahasiswa.tampilInformasi();
    }
    tampilIPKdiAtasHelper(node.right, ipkBatas);
}
}

// Tugas 3: tampilMahasiswaIPKdiAtas
System.out.println(x: "\n--- Tugas 3: tampilMahasiswaIPKdiAtas(3.50) ---");
bstRekursif.tampilMahasiswaIPKdiAtas(ipkBatas: 3.50);
}

--- Tugas 3: tampilMahasiswaIPKdiAtas(3.50) ---
Mahasiswa dengan IPK di atas 3.5:
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
macbook@fakhrun Jobsheet14_Tree %

```

3. Modifikasi class **BinaryTreeArray00** di atas, dan tambahkan :

- method **add(Mahasiswa data)** untuk memasukkan data ke dalam binary tree
- method **traversePreOrder()**

```

// TUGAS 4
private void resize() {
    Mahasiswa02[] newArr = new Mahasiswa02[dataMahasiswa.length * 2];
    System.arraycopy(dataMahasiswa, srcPos: 0, newArr, destPos: 0, dataMahasiswa.length);
    dataMahasiswa = newArr;
}

void add(Mahasiswa02 data) {
    // Pastikan array cukup besar
    if (dataMahasiswa.length < 1) resize();
    // Jika tree kosong
    if (dataMahasiswa[0] == null) {
        dataMahasiswa[0] = data;
        idxLast = 0;
        return;
    }
    // Traversal BST: kiri jika IPK lebih kecil, kanan jika lebih besar/sama
    int idx = 0;
    while (true) {
        if (data.ipk < dataMahasiswa[idx].ipk) {
            int left = 2 * idx + 1;
            while (left >= dataMahasiswa.length) resize();
            if (dataMahasiswa[left] == null) {
                dataMahasiswa[left] = data;
                if (left > idxLast) idxLast = left;
                return;
            } else {
                idx = left;
            }
        } else {
            int right = 2 * idx + 2;
            while (right >= dataMahasiswa.length) resize();
            if (dataMahasiswa[right] == null) {
                dataMahasiswa[right] = data;
                if (right > idxLast) idxLast = right;
                return;
            } else {
                idx = right;
            }
        }
    }
}

void traversePreOrder(int idxStart) {
    if (idxStart <= idxLast) {
        if (dataMahasiswa[idxStart] != null) {
            dataMahasiswa[idxStart].tampilInformasi();
            traversePreOrder(2 * idxStart + 1);
            traversePreOrder(2 * idxStart + 2);
        }
    }
}
}

```

```
// TUGAS 4: method add() dan traversePreOrder()
System.out.println(x: "\n===== Tugas 4: method add() dan traversePreOrder() =====");
BinaryTreeArray02 bta2 = new BinaryTreeArray02();

bta2.add(new Mahasiswa02(nim: "244160121", nama: "Ali", kelas: "A", ipk: 3.57));
bta2.add(new Mahasiswa02(nim: "244160221", nama: "Badar", kelas: "B", ipk: 3.85));
bta2.add(new Mahasiswa02(nim: "244160185", nama: "Candra", kelas: "C", ipk: 3.21));
bta2.add(new Mahasiswa02(nim: "244160220", nama: "Dewi", kelas: "B", ipk: 3.54));
bta2.add(new Mahasiswa02(nim: "244160131", nama: "Devi", kelas: "A", ipk: 3.72));
bta2.add(new Mahasiswa02(nim: "244160205", nama: "Ehsan", kelas: "D", ipk: 3.37));
bta2.add(new Mahasiswa02(nim: "244160170", nama: "Fizi", kelas: "B", ipk: 3.46));

System.out.println(x: "\nInOrder Traversal (urut IPK kecil ke besar):");
bta2.traverseInOrder(idxStart: 0);

System.out.println(x: "\nPreOrder Traversal (Root -> Kiri -> Kanan):");
bta2.traversePreOrder(idxStart: 0);
}
```

===== Tugas 4: method add() dan traversePreOrder() =====

InOrder Traversal (urut IPK kecil ke besar):
 NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
 NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
 NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
 NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
 NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
 NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
 NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

PreOrder Traversal (Root -> Kiri -> Kanan):
 NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
 NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
 NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
 NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
 NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
 NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
 NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
 macbook@fakhrun Jobsheet14_Tree %